



Høgskulen  
på Vestlandet



DAT108

Programmering og webapplikasjoner

F02 – Intro til lambda-uttrykk

---

Lars-Petter Helland  
Bergen  
21. august 2026

# Funksjonell programmering

Læringsutbytter (fra emneplanen):

- › Kjenne forskjellen på objektorientert og funksjonell programmering
- › Kunne bruke grunnleggende elementer fra funksjonell programmering

Vi skal holde på i ca. 2 uker med dette og avrunde med innlevering av Oblig1



# Pensum til modul om Funksjonell programmering

## **Videoer:**

- › Vi kommer til å følge omtrent samme opplegg som kurset “Java 8 Lambda Basics” fra javabrain.io, i alt ca. 20+ korte videoer åpent tilgjengelig på Youtube >>>  
<https://www.youtube.com/playlist?list=PLqq-6Pq4ITTa9YGfyhyW2CqdtW9RtY-I3>

## **Lesestoff:**

- › Vi har ikke funnet én artikkel som dekker "pensum", men det finnes mange gode ressurser. Søk på "java 8 lambda tutorial for beginners" el.l.

## **Forelesninger og forelesningsnotater:**

- › Muligens ikke så mye tekst. Kommer til å legge mest vekt på ...

## **Eksempler fra forelesningene:**

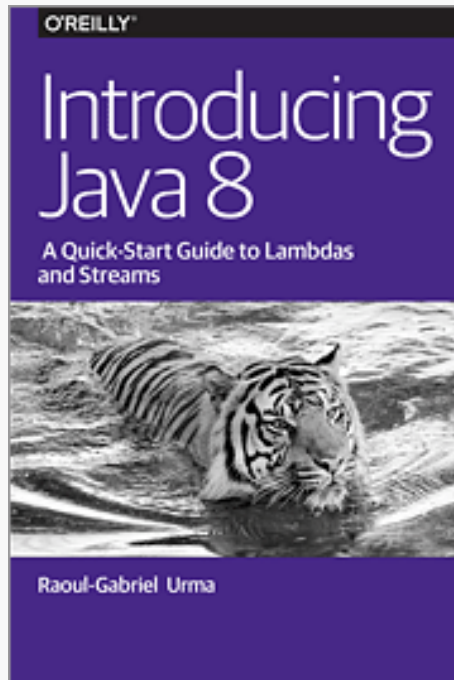
- › Det blir mye koding i timen. Alle kodeeksempler legges ut.

## **Obligatorisk oppgave1**



# Lesestoff

Kanskje <https://www.oreilly.com/programming/free/introducing-java-8.csp> er grei?



# I dag

De 12 første videoene, ca. 60 minutter:

| Navn                                                                                                                                                     | Lengde   |
|----------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
|  Java 8 Lambda Basics 1 - Introduction.mp4                              | 00:02:13 |
|  Java 8 Lambda Basics 2 - Why Lambdas.mp4                               | 00:01:46 |
|  Java 8 Lambda Basics 4 - Functional vs Object Oriented Programming.mp4 | 00:01:29 |
|  Java 8 Lambda Basics 5 - Passing Behavior in OOP.mp4                   | 00:05:47 |
|  Java 8 Lambda Basics 6 - Introducing Lambda Expressions.mp4            | 00:07:31 |
|  Java 8 Lambda Basics 7 - Lambda Expression Examples.mp4                | 00:09:50 |
|  Java 8 Lambda Basics 8 - Lambda as interface type.mp4                  | 00:09:55 |
|  Java 8 Lambda Basics 9 - Lambdas vs Interface Implementations.mp4      | 00:07:19 |
|  Java 8 Lambda Basics 10 - Type Inference.mp4                           | 00:07:16 |
|  Java 8 Lambda Basics 11 - Runnable Using Lambdas.mp4                   | 00:06:13 |
|  Java 8 Lambda Basics 12 - Functional Interface.mp4                   | 00:04:57 |



# Python-eksempelet fra sist (nå med typehint 😊)

```
def dobbel(tall: int) -> int:
    return 2 * tall

def kvadrat(tall: int) -> int:
    return tall * tall

def omform(liste: list[int], fu: Callable[[int],int]) -> list[int]:
    ny_liste = []
    for tall in liste:
        ny_liste.append(fu(tall))
    return ny_liste

listen: list[int] = [1, 2, 3, 4]

dobbelt_liste = omform(listen, dobbel)
print("dobbelt_liste: ", dobbelt_liste)

kvadrat_liste = omform(listen, kvadrat)
print("kvadrat_liste: ", kvadrat_liste)
```

Hovedbudskapet med dette eksempelet er bruken av funksjoner.

`omform` er en høyere ordens funksjon som tar en annen funksjon `fu` som parameter og bruker denne til hjelp i beregningene.

`omform` blir da den generelle algoritmen, mens omformingen av hvert enkeltelement delegeres til `fu`.

Vi ser at vi kan kalle `omform` med ulike funksjoner for `fu` (her vist med `dobbelt` og `kvadrat`) og få ulike resultat.

Vi så også (ikke vist her) at Python har en innbygget funksjon tilsvarende `omform` kalt `map`. Dette er en standard-funksjon som finnes i alle språk (inkl. Java) som tilbyr «funksjonell programmering».



# Hvordan blir det samme eksempelet i Java?

- › **Java** er et objekt-orientert språk som **ikke har direkte støtte for funksjoner** slik Python, JavaScript, Haskell m.fl. har.
- › **Hvordan gjøres dette da i Java?**



# Funksjoner i Java

- › Prosjekt: f02-sortering-utplukk
- › Pakke: no.hv1.dat108.funksjoner
- › Tar utgangspunkt i f02-sortering-utplukk-fyllinn
- › Løsning legges ut i <https://github.com/lars-petter-helland/dat108-h2026> etter timen



# Oppsummering fra Java-eksempelet

- › Siden Java ikke har direkte støtte for funksjoner, men er objektorientert, vil **funksjoner i Java være representert som objekter**.
- › Vi trenger et interface for å representere den generelle funksjonstypen (her `int -> int`):
- › Vi trenger en klasse for å definere den aktuelle funksjonstypen for funksjonen vi ønsker:
- › Et funksjonsobjekt av denne typen kan sendes som parameter til en annen funksjon/metode:

```
interface HeltallsFunksjon {  
    int av(int x);  
}
```

```
class Dobbel implements HeltallsFunksjon {  
    @Override  
    public int av(int x) {  
        return 2*x;  
    }  
}
```

```
static List<Integer> omform(List<Integer> liste, HeltallsFunksjon fu) {  
    //...
```

```
List<Integer> dobbelliste = omform(listen, new Dobbel());
```

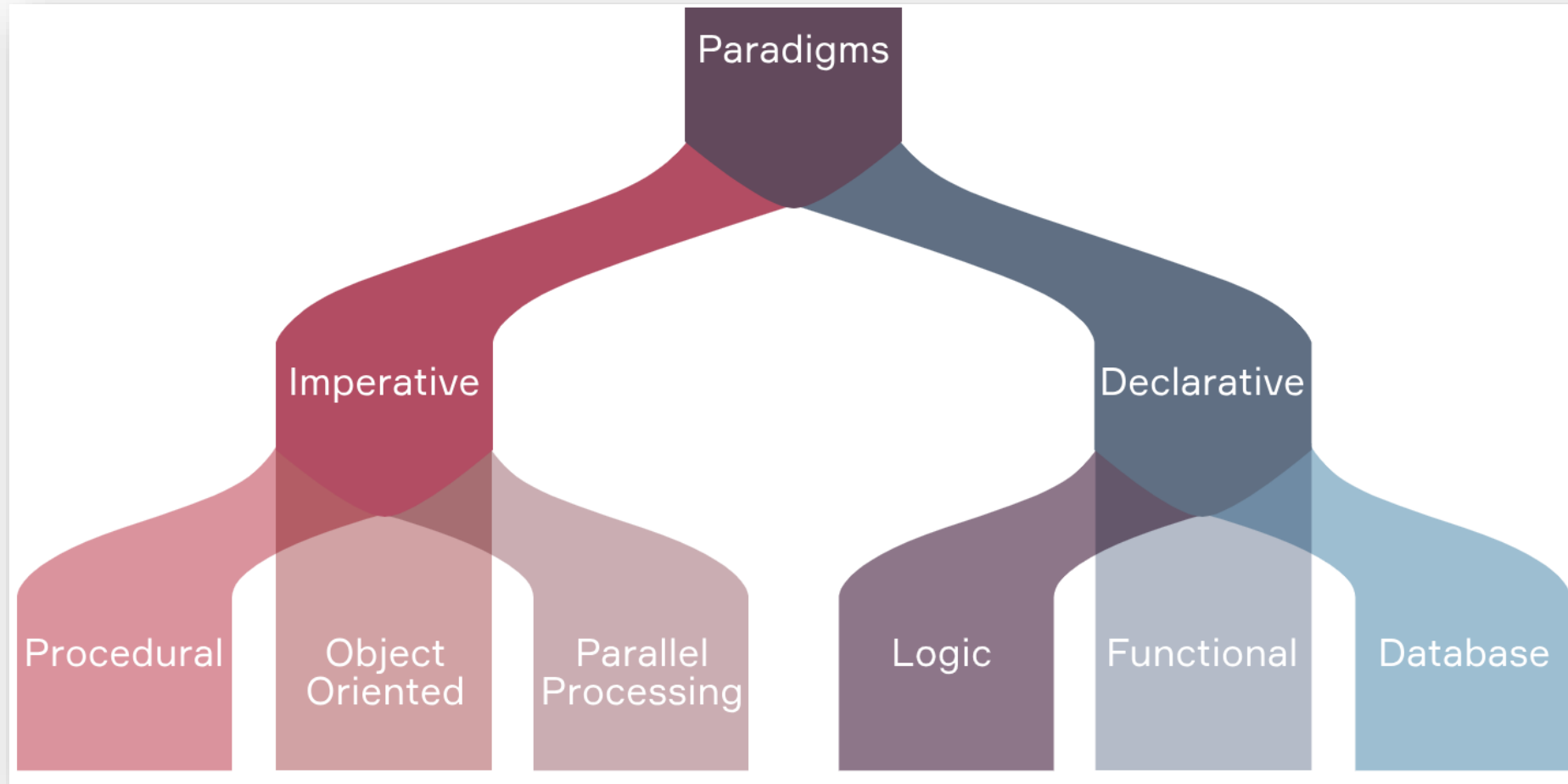


# Nå

- › Det å jobbe med **samlinger av data** er veldig vanlig i databehandling, og vi ønsker derfor å kunne programmere dette på en god og fleksibel måte.
- › Vi skal se hvordan vi kan bruke en funksjonell programmeringsstil for å få dette til, og vi skal se på de ulike byggeklossene og tenkemåtene som utgjør **funksjonell programmering**.



# Programmeringsparadigmer (-stiler)



# Imperativ vs. funksjonell programmering

- › **Imperativ stil:** En sekvens av setninger som via endringer i programmets tilstand (variabler) arbeider seg frem til et svar.

```
List<Integer> positiveTall = new ArrayList<>();  
for (int x : enListeMedTall) {  
    if (x > 0) {  
        positiveTall.add(x);  
    }  
}
```

- › **Funksjonell stil:** Nøstede funksjonskall som gir et svar direkte.

```
List<Integer> positiveTall = enListeMedTall.filter(x -> x > 0);
```



Funksjon / Lambda-uttrykk

# Hva er lambda-uttrykk ?

- › Siden det var nevnt på forrige slide:
- › Begrepet lambda-uttrykk er hentet fra "lambda calculus" der lambda-symbolet brukes til å angi en **anonym funksjon**.

```
f(x) = x2 + 2           // Matematisk  
λx.x2 + 2               // Lambda calculus  
lambda x: x * x + 2     // Python  
x => x * x + 2          // JavaScript  
x -> x * x + 2         // Java
```

- › Slike kommer dere til å se mye !
- › Vi skal se at språkstøtte for **anonyme funksjoner** (funksjonslitteraler) er en egenskap som oppmuntrer til en **funksjonell programmeringsstil**, som passer godt når vi jobber med samlinger av data.



# Å gjøre en algoritme fleksibel - Utvalgssortering

- › GitHub: <https://github.com/lars-petter-helland/dat108-h2026>
- › **f02-sortering-utplukk-fyllinn**
- › --- no.hvl.dat108.sortering1
- › Utvalgssortering på liste av heltall
- › Utvalgssortering på liste av String-objekter
- › Utvalgssortering som virker både på Integer og String
- › Utvalgssortering på liste av Person-objekter
- › Hva hvis vi ønsker å sortere på en annen egenskap?
- › Etter timen legges det ut et løsningsprosjekt i GitHub med navnet **f02-sortering-utplukk-losning**.





# Java Collections.sort(...)

- › GitHub: <https://github.com/lars-petter-helland/dat108-h2026>
- › **f02-sortering-utplukk-fyllinn**
- › --- no.hvl.dat108.collsort
- › Sortering av strenger. Sortere med "naturlig" rekkefølge.
- › Sortere alfabetisk (uavhengig av små og store bokstaver)
- › Bruke anonym indre klasse.
- › Hvordan lambda-uttrykk kan forenkle koden betraktelig
- › Sortere på strenglengde
- › Navngi lambda-uttrykk med variabler
- › Etter timen legges det ut et løsningsprosjekt i GitHub med navnet **f02-sortering-utplukk-losning**.



# Hva oppnådde vi ved å bruke et lambda-uttrykk?

- › Med lambda-uttrykk kan små kodesnutter sendes rundt i programmet, f.eks. som parameter til en metode. Vi sier at vi kan "parametrisere" oppførselen.
- › Vi får også fjernet en del av det man kaller "boilerplate-kode"
- › Vi får mer **lesbar og konsis kode**, og **færre kodelinjer**.
- › Med trening/erfaring vil vi begynne å skrive kode på et **høyere abstraksjonsnivå**.
- › Med trening/erfaring vil vi begynne å skrive kode som er mer **funksjonell i stedet for imperativ**.



# Å plukke ut elementer fra en liste

- › GitHub: <https://github.com/lars-petter-helland/dat108-h2026>
- › **f02-sortering-utplukk-fyllinn**
- › --- no.hvl.dat108.**utplukk**
- › Plukke ut tall fra liste. Først se på hvordan vi kan skrive en utplukker.
- › Hvordan kan vi la betingelsen være parameter
- › Løsning uten lambda-uttrykk
- › Løsning med lambda-uttrykk forenkler koden betraktelig
- › Parametrisering av lambda-uttrykket? (Neste gang?)
- › Lage en metode som returnerer et "dynamisk" lambda-uttrykk? (Neste gang?)
- › Etter timen legges det ut et løsningsprosjekt i GitHub med navnet **f02-sortering-utplukk-losning**.



# Hva klarte vi å gjøre nå?

- › Vi skrev en metode som tar en funksjon (lambda) som input. Vi kan da parametrisere oppførsel og bygge mer generelle metoder.
- › Vi så hvordan vi selv kan definere en "funksjonell" kontrakt (et interface med én metode) og bruke dette som parameter i en annen metode.
- › Vi så hvordan vi kan lage en metode som returnerer en funksjon (lambda). Med denne teknikken kan man bygge mer generelle funksjoner som kan gjenbrukes i programmene våre. (Neste gang?)



# Lambda-uttrykk, noen eksempler

```
(int x) -> 2 * x           // Returnerer en int
(a, b) -> (a + b)/2.0     // Returnerer en double
s -> System.out.println(s) // Returnerer ingenting (void)
() -> System.out.println("Hallo") // Returnerer ingenting (void)

(String s) -> {           // Returnerer en String
    if (s != null && s.equals("Bingo")) {
        System.out.println(s);
    }
    return s.toUpperCase();
}
```



# Lambda-uttrykk, generell syntaks

`() -> uttrykk` (Hvis det er ingen parametre)

`parameter -> uttrykk` (Hvis det kun er én parameter)

`(parametre) -> uttrykk` (Hvis det kun er én kodelinje)

`(parametre) -> {kodelinjer}` (Evt. siste linje return ...;)

...



# Trenger vi egentlig funksjonell programmering?

- › Vi klarer oss godt uten, men vi skal se noen fordeler. Vi får:
  - › Kode som er **mer fleksibel** (vi kan "parametrisere" oppførsel)
  - › Kode som er **mer lesbar** (høyere abstraksjonsnivå, færre kodelinjer)
  - › Kode som er **lettere å vedlikeholde** (færre kodelinjer, bedre modularitet)
  - › Kode som er **lettere å teste** (færre variabler, ingen sideeffekter)
  - › Kode som er **lettere å parallelisere** (kommer tilbake til det)



# Funksjonell kontrakt

En funksjonell kontrakt (functional interface) er en kontrakt (interface) med kun én (abstrakt) metode.

Annoteres med **@FunctionalInterface**.

```
@FunctionalInterface
interface MathOperation {
    int operate(int a, int b); // Exactly one abstract method
}
```

Annotasjonen **@FunctionalInterface** er for å gi utvikleren beskjed om at dette interfacet kun skal ha én metode, og at endring på dette vil ødelegge for eksisterende kode (lambda-uttrykk).



# Siste programeksempel

Nå kan dere prøve dere litt selv (evt. gjøre til mandag)

Lag en generell metode **betingetUtskrift(...)** som tar inn en streng og et (false | true) lambda-uttrykk og som skriver ut strengen kun hvis uttrykket er sant.

Lag en **main(...)**-metode som tester dette ut med et par ulike input, f.eks. krav om at stringen begynner på "X" eller at den er lengre enn 3 tegn.



# "Løsning" på repetisjonsoppgavene dere fikk

- › Det blir lagt ut et løsningsforslag søndag ettermiddag.
- › Hvis dere ikke allerede har gjort oppgavene, så gjør i hvert fall Oppgave 1 og Oppgave 2 denne uken.
- › *Det er lagt ut flere oppgaver i dag (se neste slide), som dere kan jobbe med på labben.*
- › *Neste uke blir Oblig1 lagt ut!*



# Videre

## Lab-oppgaver: Oppgaver-uke34\_456 - Funksjoner og lambda-uttrykk

Neste gang mandag:

|                                                                                   |                                                              |          |
|-----------------------------------------------------------------------------------|--------------------------------------------------------------|----------|
|  | Java 8 Lambda Basics 13 - Lambda Exercise.mp4                | 00:04:04 |
|  | Java 8 Lambda Basics 14 - Lambda Exercise Solution.mp4       | 00:17:31 |
|  | Java 8 Lambda Basics 15 - Using Function Interfaces.mp4      | 00:07:05 |
|  | Java 8 Lambda Basics 16 - More Functional Interfaces.mp4     | 00:06:22 |
|  | Java 8 Lambda Basics 19 - Closures in Lambda Expressions.mp4 | 00:08:29 |
|  | Java 8 Lambda Basics 21 - Method References.mp4              | 00:10:26 |

Deretter:

- › Dere starter på Oblig1 (lambda-uttrykk ++)
- › Streams
- › Funksjonell programmering